

Problem 1

Need to minimize the following:

$$C(\boldsymbol{\omega}) = -\frac{1}{N}L(\boldsymbol{\omega}) + \lambda \boldsymbol{\omega}^T \boldsymbol{\omega}$$

where

$$L(\boldsymbol{\omega}) = \sum_{j=1}^n y_j(\omega_0 + \sum_{i=1}^p \omega_i x_{ji}) - \ln(1 + \exp(\omega_0 + \sum_{i=1}^p \omega_i x_{ji}))$$

We minimize this loss using gradient descent, using:

$$\boldsymbol{w}^{t+1} \leftarrow \boldsymbol{w}^t - \eta \lambda \boldsymbol{w}^t + \frac{\eta}{N} \frac{\partial L(\boldsymbol{w}^t)}{\partial \boldsymbol{w}}$$

Let the linear predictor for observation j be

$$z_j = w_0 + \sum_{i=1}^p w_i x_{ji},$$

and define the augmented covariate vector $\tilde{\boldsymbol{x}}_j = (1, x_{j1}, \dots, x_{jp})^T$ and the parameter vector $\boldsymbol{\omega} = (w_0, w_1, \dots, w_p)^T$. The log-likelihood can then be written as

$$L(\boldsymbol{\omega}) = \sum_{j=1}^N [y_j z_j - \ln(1 + e^{z_j})].$$

Since $\partial z_j / \partial \boldsymbol{\omega} = \tilde{\boldsymbol{x}}_j$, applying the chain rule yields

$$\frac{\partial L(\boldsymbol{\omega})}{\partial \boldsymbol{\omega}} = \sum_{j=1}^N \left(y_j - \frac{e^{z_j}}{1 + e^{z_j}} \right) \tilde{\boldsymbol{x}}_j.$$

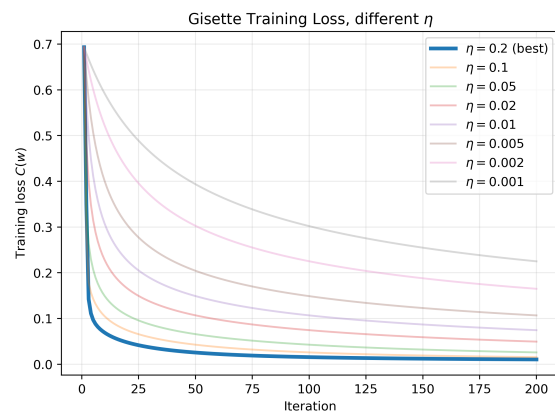
Stacking terms gives the gradient in vector form,

$$\nabla_{\boldsymbol{\omega}} L(\boldsymbol{\omega}) = \tilde{\boldsymbol{X}}^T \left(\boldsymbol{y} - \frac{e^{\boldsymbol{z}}}{1 + e^{\boldsymbol{z}}} \right),$$

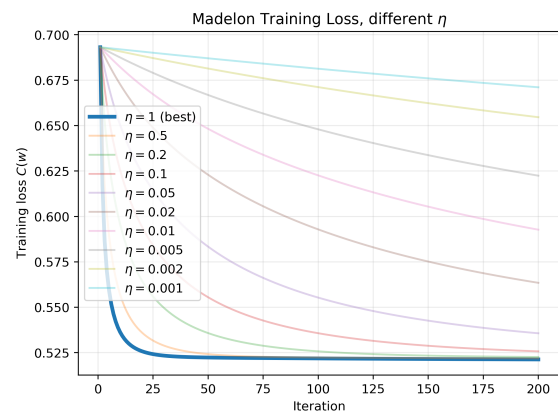
where $\tilde{\boldsymbol{X}}$ is the augmented design matrix with rows $\tilde{\boldsymbol{x}}_j^T$ and $\boldsymbol{z} = \tilde{\boldsymbol{X}} \boldsymbol{\omega}$.

- For each of the three datasets, we loop over a grid to find the best η (in terms of smallest final training loss), and obtain the plots shown in figure 1. We then report the misclassification error rates for the train and test datasets for the three datasets. This is reported in 1.

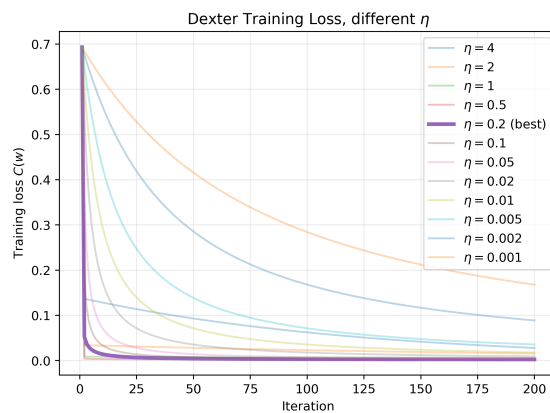
Lastly, we are asked to plot the ROC curve of both the training and test sets. We do so in figure 2



(a) Gisette: training loss vs. iteration for multiple η values.



(b) Madelon: training loss vs. iteration for multiple η values.

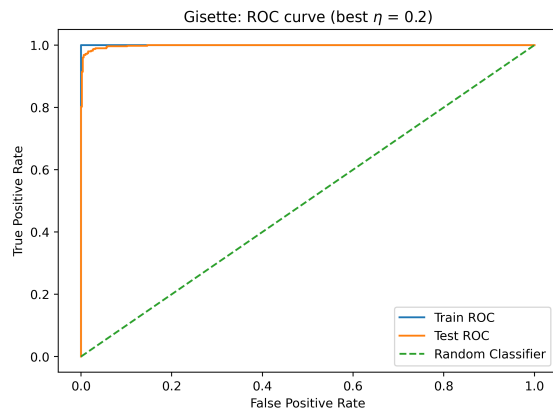


(c) Dexter: training loss vs. iteration for multiple η values.

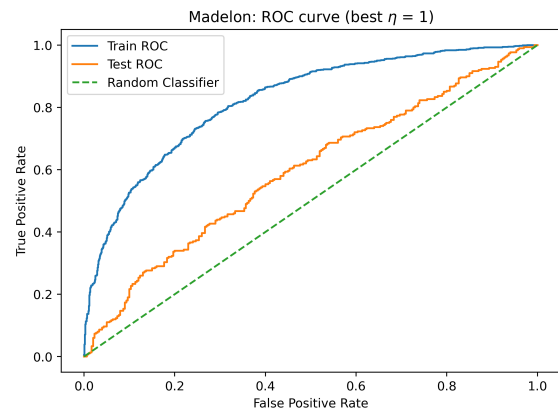
Figure 1: Training losses vs. iteration for logistic regression across possible learning rates η .

dataset	best eta	train misclass	test misclass
Gisette	0.200000	0.000000	0.020000
Madelon	1.000000	0.259500	0.426667
Dexter	0.200000	0.000000	0.140000

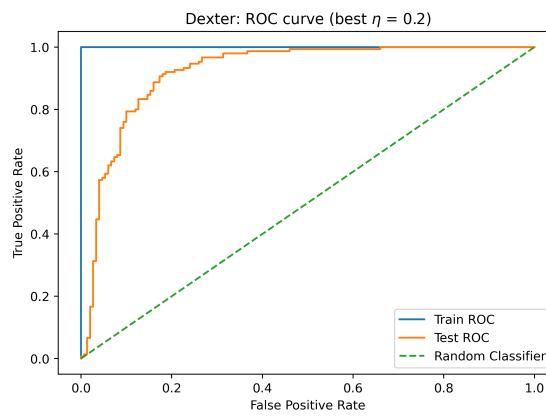
Table 1: Best learning rate η , training and test misclassification error, Logistic Regression.



(a) Gisette: training and test ROC curves for best η .



(b) Madelon: training and test ROC curves for best η .



(c) Dexter: training and test ROC curves for best η .

Figure 2: ROC curves for logistic regression on the training and test sets for each dataset.

Problem 2

We need to minimize analytically:

$$C(\boldsymbol{\omega}) = \frac{1}{N} \sum_{i=1}^n (y_i - \boldsymbol{\omega}^T \mathbf{x}_i - \omega_0)^2 + \lambda (\boldsymbol{\omega}^T \boldsymbol{\omega} + \omega_0^2)$$

where $\lambda = 0.001$, $(\mathbf{x}_i, y_i)$ are the training data, and $y_i \in \{-1, +1\}$. Predict classification via $\hat{y} = \text{sign}(\boldsymbol{\omega}^T \mathbf{x} + \omega_0)$.

If we define the augmented covariate vector and parameter vector as in question 1, then we can write the minimization function as

$$C(\tilde{\boldsymbol{\omega}}) = \frac{1}{N} \|\mathbf{y} - \tilde{\mathbf{X}} \tilde{\boldsymbol{\omega}}\|^2 + \lambda \|\tilde{\boldsymbol{\omega}}\|^2.$$

$$C(\boldsymbol{\omega}) = \frac{1}{N} \left(\mathbf{y}^T \mathbf{y} - 2 \mathbf{y}^T \tilde{\mathbf{X}} \tilde{\boldsymbol{\omega}} + \tilde{\boldsymbol{\omega}}^T \tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \tilde{\boldsymbol{\omega}} \right) + \lambda \tilde{\boldsymbol{\omega}}^T \tilde{\boldsymbol{\omega}}$$

We can take derivatives with respect to $\tilde{\boldsymbol{\omega}}$ and setting equal to 0:

$$\nabla_{\boldsymbol{\omega}} C(\boldsymbol{\omega}) = \frac{1}{N} \left(-2 \tilde{\mathbf{X}}^T \mathbf{y} + 2 \tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \tilde{\boldsymbol{\omega}} \right) + 2 \lambda \tilde{\boldsymbol{\omega}}$$

$$0 = 2 \left(\frac{1}{N} \left(\tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \tilde{\boldsymbol{\omega}} - \tilde{\mathbf{X}}^T \mathbf{y} \right) + \lambda \tilde{\boldsymbol{\omega}} \right)$$

$$0 = \frac{1}{N} \left(\tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \tilde{\boldsymbol{\omega}} - \tilde{\mathbf{X}}^T \mathbf{y} \right) + \lambda \tilde{\boldsymbol{\omega}}$$

$$0 = \tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \tilde{\boldsymbol{\omega}} - \tilde{\mathbf{X}}^T \mathbf{y} + \lambda N \tilde{\boldsymbol{\omega}}$$

$$\left(\tilde{\mathbf{X}}^T \tilde{\mathbf{X}} + \lambda N I \right) \tilde{\boldsymbol{\omega}} = \tilde{\mathbf{X}}^T \mathbf{y}$$

where I is the $(p+1) \times (p+1)$ identity matrix (for matrix multiplication). We can use Python linear algebra solvers to find a solution to this (and we do), but the analytic solution would be:

$$\hat{\tilde{\boldsymbol{\omega}}} = \left(\tilde{\mathbf{X}}^T \tilde{\mathbf{X}} + \lambda N I \right)^{-1} \tilde{\mathbf{X}}^T \mathbf{y}$$

- We can report the misclassification error of the training and test datasets in the following table:

train misclass error	test misclass error
0.000000	0.091000

Table 2: misclassification Error for training and test datasets, Ridge like regression

The train misclassification error rate is the same (0.00), though the test error rate is higher (0.091 vs. 0.020).

- We can display the ROC curve for both the ridge and logistic models in figure 3. The train ROC for the logistic and ridge models are practically the same. The ROC for the logistic model is better than the ridge though; tighter to the upper left hand corner.

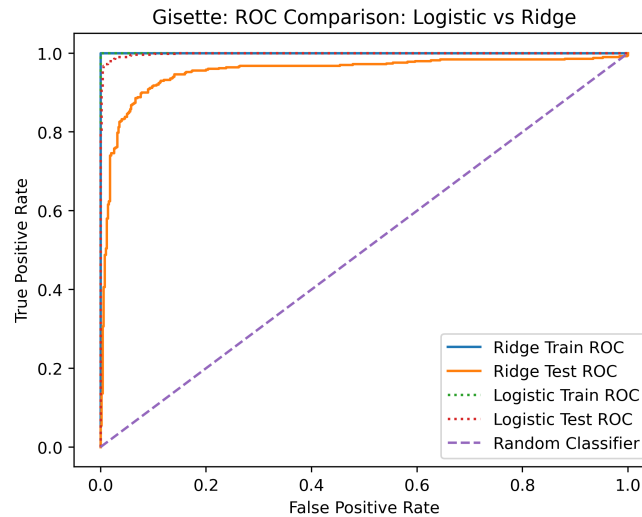


Figure 3: ROC curves for ridge and logistic regression models.

Code

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.metrics import roc_curve
5 import matplotlib.pyplot as plt
6
7 datasets = [
8     {
9         "name": "Gisette",
10        "X_train": "assignments/hw03/data/gisette_train.data",
11        "y_train": "assignments/hw03/data/gisette_train.labels",
12        "X_valid": "assignments/hw03/data/gisette_valid.data",
13        "y_valid": "assignments/hw03/data/gisette_valid.labels",
14        "eta_out": "assignments/hw03/figures/gisette_eta_plot.png",
15        "roc_out": "assignments/hw03/figures/gisette_roc_plot.png",
16    },
17    {
18        "name": "Madelon",
19        "X_train": "assignments/hw03/data/madelon_train.data",
20        "y_train": "assignments/hw03/data/madelon_train.labels",
21        "X_valid": "assignments/hw03/data/madelon_valid.data",
22        "y_valid": "assignments/hw03/data/madelon_valid.labels",
23        "eta_out": "assignments/hw03/figures/madelon_eta_plot.png",
24        "roc_out": "assignments/hw03/figures/madelon_roc_plot.png",
25    },
26    {
27        "name": "Dexter",

```

```
28     "X_train": "assignments/hw03/data/dexter_train.csv",
29     "y_train": "assignments/hw03/data/dexter_train.labels",
30     "X_valid": "assignments/hw03/data/dexter_valid.csv",
31     "y_valid": "assignments/hw03/data/dexter_valid.labels",
32     "eta_out": "assignments/hw03/figures/dexter_eta_plot.png",
33     "roc_out": "assignments/hw03/figures/dexter_roc_plot.png",
34 }
35 ]
36
37 #assignment specific parameters + number of iterations + some possible learning
38 rates
39 lamb = 0.001
40 iters = 200
41 eta_vals = [4, 2, 1, 0.5, 0.2, 0.1, 0.05, 0.02, 0.01, 0.005, 0.002, 0.001]
42
43 #store final results for the summary table + store the ROC curves
44 results = []
45 roc_store = {}
46
47 #Q1
48 #loop over datasets
49 for d in datasets:
50     #get name of dataset
51     name = d["name"]
52     print(name)
53
54     #Load datasets. use default read.csv if Dexter dataset
55     if name == "Dexter":
56         X_train = pd.read_csv(d["X_train"], header=None)
57         X_valid = pd.read_csv(d["X_valid"], header=None)
58     else:
59         X_train = pd.read_csv(d["X_train"], delim_whitespace=True, header=None)
60         X_valid = pd.read_csv(d["X_valid"], delim_whitespace=True, header=None)
61
62     y_train = pd.read_csv(d["y_train"], header=None).values.ravel()
63     y_valid = pd.read_csv(d["y_valid"], header=None).values.ravel()
64
65     #convert labels to 0/1 for likelihood to work
66     y_train_converted_01 = (y_train == 1).astype(int)
67     y_valid_converted_01 = (y_valid == 1).astype(int)
68
69     #get N and p
70     N, p = X_train.shape
71
72     #standardize
73     scaler = StandardScaler()
74     X_train_scaled = scaler.fit_transform(X_train)
```

```

74 X_test_scaled = scaler.transform(X_valid)
75
76 #add column of 1s to augment w
77 X_train_tilde = np.column_stack([np.ones(len(X_train_scaled)),
X_train_scaled])
78 X_valid_tilde = np.column_stack([np.ones(len(X_test_scaled)), X_test_scaled])
79
80 #store all loss curves for this dataset
81 #dict of loss curves, store best learning rate, and smallest final loss
82 #plus coefficient vector of best w from best eta
83 loss_curves = {}
84 best_eta = None
85 best_final = np.inf
86 best_w = None
87
88 #loop over the eta values
89 for eta in eta_vals:
90     #set everything to 0 initially for each eta
91     w = np.zeros(p+1)
92     losses = []
93
94     #checks for if loss is decreasing at each iteration
95     prev = np.inf
96     monotone = True
97
98     print(f"{eta}")
99     #loop over the iterations
100    for t in range(iters):
101        #get predictor
102        z = X_train_tilde @ w
103
104        #compute log likelihood
105        #z = linear algebra product of X and w, with column of ones added
106        #https://numpy.org/devdocs/reference/generated/numpy.logaddexp.html
107        L = np.sum(y_train_converted_01*z - np.logaddexp(0.0, z))
108
109        #Loss function.
110        C = ((-1.0/N)*L) + (lamb*np.dot(w, w))
111        losses.append(C)
112
113        #checking to see if the loss is monotone decreasing
114        if t > 0 and C > prev + 1e-12:
115            monotone = False
116            break
117        prev = C
118
119    #get the gradient of the log likelihood

```

```
120         exponential_term = np.exp(-np.logaddexp(0.0, -z))
121         gradient_L = X_train_tilde.T @ (y_train_converted_01 -
exponential_term)
122
123         #update w
124         w = w - eta*lamb*w + (eta/N)*gradient_L
125
126         #check if the series of loss scores is monotone decreasing and that the
127         #length is correct
128         if monotone and len(losses) == iters:
129             #save full curve
130             losses = np.array(losses)
131             loss_curves[eta] = losses
132
133             #compare loss at final iteration to the best loss value so far
134             #if it is smaller, update best loss, best eta, and best w vector.
135             if losses[-1] < best_final:
136                 best_final = losses[-1]
137                 best_eta = eta
138                 best_w = w.copy()
139
140         #figure showing Loss vs iteration
141         plt.figure()
142
143         for eta, curve in loss_curves.items():
144             #if we have the best eta, make it thicker and less transparent
145             #some nicer labels
146             if eta == best_eta:
147                 plt.plot(np.arange(1, iters + 1), curve, linewidth=3, alpha=1.0,
label=rf"$\eta={eta}$ (best)")
148             else:
149                 plt.plot(np.arange(1, iters + 1), curve, linewidth=1.5, alpha=0.3,
label=rf"$\eta={eta}$")
150
151         #saves plot to folder
152         plt.xlabel("Iteration")
153         plt.ylabel("Training Loss  $C(w)$ ")
154         plt.title(rf"{name}_Training Loss, different  $\eta$ ")
155         plt.legend()
156         plt.grid(alpha=0.25)
157         plt.tight_layout()
158         plt.savefig(d["eta_out"], dpi=400, bbox_inches="tight")
159         plt.close()
160         #plt.show()
161
162         #making predictions
163         #making predictions for training set
```



```

164 z_train = X_train_tilde @ best_w
165 z_test = X_valid_tilde @ best_w
166
167 #get probabilities for 1 and 0
168 p_train_1 = np.exp(-np.logaddexp(0.0, -z_train))
169 p_train_0 = np.exp(-np.logaddexp(0.0, z_train))
170
171 p_test_1 = np.exp(-np.logaddexp(0.0, -z_test))
172 p_test_0 = np.exp(-np.logaddexp(0.0, z_test))
173
174 #compute the odds of train and test
175 odds_train = p_train_1/p_train_0
176 odds_test = p_test_1/p_test_0
177
178 #make predictions. Predcit 1 if Odds > 1
179 #as.int makes things nicer
180 yhat_train = (odds_train > 1.0).astype(int)
181 yhat_test = (odds_test > 1.0).astype(int)
182
183 #misclassification error
184 train_err = np.mean(yhat_train != y_train_converted_01)
185 test_err = np.mean(yhat_test != y_valid_converted_01)
186
187 #get ROC curves
188 fpr_train, tpr_train, thresholds_train = roc_curve(y_train, p_train_1)
189 fpr_test, tpr_test, thresholds_test = roc_curve(y_valid, p_test_1)
190
191 #save Gisette data for Q2
192 if name == "Gisette":
193     roc_store[name] = {
194         "train": (fpr_train, tpr_train),
195         "test": (fpr_test, tpr_test)
196     }
197
198 #plot these and save in figures folder
199 plt.figure()
200 plt.plot(fpr_train, tpr_train, label="Train_ROC")
201 plt.plot(fpr_test, tpr_test, label="Test_ROC")
202 plt.plot([0, 1], [0, 1], linestyle="--", label="Random_Classifier")
203 plt.xlabel("False_Positive_Rate")
204 plt.ylabel("True_Positive_Rate")
205 plt.title(rf"{d['name']}: ROC curve_{best_\eta_{best_eta}}")
206 plt.legend()
207 plt.tight_layout()
208 plt.savefig(d["roc_out"], dpi=400, bbox_inches="tight")
209 plt.close()
210 #plt.show()

```

```
211
212     #need to add results
213     results.append({
214         "dataset": name,
215         "best_eta": best_eta,
216         "train_misclass": train_err,
217         "test_misclass": test_err,
218     })
219
220 #print results table
221 results_df = pd.DataFrame(results)
222 LATEX_table = results_df.to_latex(
223     index=False,
224     caption=None,
225     label=None,
226     column_format="c" * results_df.shape[1],
227     escape=False
228 )
229
230 #save latex table
231 with open("assignments/hw03/output/logreg_misclass_table.tex", "w") as f:
232     f.write(LATEX_table)
233
234 #question 2
235 #only need some of the datasets in the dict
236 selected_names = {"Gisette"}
237 filtered_datasets = list(filter(lambda da: da["name"] in selected_names,
238                                datasets))
239
240 for d in filtered_datasets:
241     print(f"{d['name']}")
242
243     X_train = pd.read_csv(d["X_train"], delim_whitespace=True, header=None)
244     X_valid = pd.read_csv(d["X_valid"], delim_whitespace=True, header=None)
245
246     y_train = pd.read_csv(d["y_train"], header=None).values.ravel()
247     y_valid = pd.read_csv(d["y_valid"], header=None).values.ravel()
248
249     #get shape
250     N, p = X_train.shape
251
252     #add column of 1s
253     X_train_tilde = np.column_stack([np.ones(len(X_train)), X_train])
254     X_valid_tilde = np.column_stack([np.ones(len(X_valid)), X_valid])
255
256     #identity matrix needed for analytical solution
257     I = np.eye(p+1)
```

```
257
258 #matrix math
259 LHS = X_train_tilde.T @ X_train_tilde + (lamb*N*I)
260 RHS = X_train_tilde.T @ y_train
261
262 #analytical minimizer of the penalized squared loss function
263 w_hat = np.linalg.solve(LHS, RHS)
264
265 #predictions make using  $w^T @ x$ 
266 train_score = X_train_tilde @ w_hat
267 test_score = X_valid_tilde @ w_hat
268
269 #need to get sign function
270 yhat_train = np.sign(train_score)
271 yhat_test = np.sign(test_score)
272
273 # misclassification error
274 train_misclass = np.mean(yhat_train != y_train)
275 test_misclass = np.mean(yhat_test != y_valid)
276
277 #get this into a latex table?
278 results_df = pd.DataFrame([
279     "train_misclass_error": train_misclass,
280     "test_misclass_error": test_misclass,
281 ])
282
283 LATEX_table = results_df.to_latex(
284     index=False,
285     caption=None,
286     label=None,
287     column_format="c" * results_df.shape[1],
288     escape=False
289 )
290
291 #save LATEX table
292 with open("assignments/hw03/output/q2_table.tex", "w") as f:
293     f.write(LATEX_table)
294
295 #ROC curves
296
297 #https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc\_curve.html
298 fpr_train, tpr_train, thresholds_train = roc_curve(y_train, train_score)
299 fpr_test, tpr_test, thresholds_test = roc_curve(y_valid, test_score)
300
301 #need to find a way to plot this and the result from Q1
302 plt.figure()
303 plt.plot(fpr_train, tpr_train, label="Ridge_Train_ROC")
```

```
303 plt.plot(fpr_test, tpr_test, label="Ridge_Test_ROC")
304
305 #need to overlay the Q1 Gisette portion
306 fpr_train_q1, tpr_train_q1 = roc_store["Gisette"]["train"]
307 fpr_test_q1, tpr_test_q1 = roc_store["Gisette"]["test"]
308
309 plt.plot(fpr_train_q1, tpr_train_q1, linestyle=":", label="Logistic_Train_
ROC")
310 plt.plot(fpr_test_q1, tpr_test_q1, linestyle=":", label="Logistic_Test_ROC")
311
312 plt.plot([0, 1], [0, 1], linestyle="--", label="Random_Classifier")
313 plt.xlabel("False_Positive_Rate")
314 plt.ylabel("True_Positive_Rate")
315 plt.title(f"{d['name']}: ROC_Comparison: Logistic_vs_Ridge")
316 plt.legend()
317 plt.savefig("assignments/hw03/figures/q2_roc_plot.png", dpi=400,
bbox_inches="tight")
318 plt.close()
319 #plt.show()
```